

AgentCore Runtime

To Begin with the Lab

Summary of the Lab

This lab demonstrates how to integrate a CrewAI-based Vacation Planner application with AWS Bedrock AgentCore Runtime. It begins by updating the `crew.py` file to initialize `BedrockAgentCoreApp`, create an entry point, process user requests, execute CrewAI workflows, and run the runtime server locally. The lab then covers creating dependencies, configuring Docker, building and pushing container images to Amazon ECR, and hosting the agent in Bedrock AgentCore. It further explains testing endpoints, creating Lambda functions to invoke the agent, exposing it through API Gateway, and finally developing a Streamlit frontend that communicates with the deployed API, enabling users to generate vacation plans interactively.

- Prerequisite for the Lab
 - VS Code
 - Python
 - AWS Account
 - AWS CLI
 - IAM credentials configured for AWS CLI
 - Docker Desktop
- This is the continuation of the previous lab
- Here we will update the `crew.py` file.
- To integrate CrewAI with AgentCore, first import and initialize `BedrockAgentCoreApp`, then define an `@app.entrypoint` function to handle incoming requests. Extract the user input from the payload, create and initialize the VacationPlanner crew, and execute the workflow using `crew.kickoff()`. Add logging and exception handling to improve debugging and reliability. Finally, start the AgentCore Runtime server with `app.run()` to enable local execution and testing.

```
from crewai import Agent, Crew, Process, Task

from crewai.project import CrewBase, agent, crew, task

from crewai.agents.agent_builder.base_agent import BaseAgent

from typing import List

from crewai_tools import SerperDevTool

import os

from crewai import LLM

# ----- #1 Agentcore imports -----
```

```

from bedrock_agentcore.runtime import BedrockAgentCoreApp

app = BedrockAgentCoreApp()


#Initialize SerperDev Tool

serper_dev_tool=SerperDevTool(api_key="08095acb4896eb2717865736cc968dc12f471d47"
)

llm=LLM(model="bedrock/us.amazon.nova-pro-v1:0")

# If you want to run a snippet of code before or after the crew starts,
# you can use the @before_kickoff and @after_kickoff decorators
# https://docs.crewai.com/concepts/crews#example-crew-class-with-decorators


@CrewBase
class VacationPlanner():
    """VacationPlanner crew"""

    # Learn more about YAML configuration files here:
    # Agents: https://docs.crewai.com/concepts/agents#yaml-configuration-recommended
    # Tasks: https://docs.crewai.com/concepts/tasks#yaml-configuration-recommended
    agents_config = 'config/agents.yaml'
    tasks_config = 'config/tasks.yaml'

    # If you would like to add tools to your agents, you can learn more about it here:
    # https://docs.crewai.com/concepts/agents#agent-tools

    @agent
    def vacation_researcher(self) -> Agent:
        return Agent(

```

```
        config=self.agents_config['vacation_researcher'],  
        verbose=True,  
        tools=[serper_dev_tool],  
        llm=llm  
    )
```

@agent

def itinerary_planner(self) -> Agent:

```
    return Agent(  
        config=self.agents_config['itinerary_planner'],  
        verbose=True,  
        llm=llm  
    )
```

To learn more about structured task outputs,

task dependencies, and task callbacks, check out the documentation:

<https://docs.crewai.com/concepts/tasks#overview-of-a-task>

@task

def research_task(self) -> Task:

```
    return Task(  
        config=self.tasks_config['research_task'],  
    )
```

@task

def reporting_task(self) -> Task:

```
    return Task(  
        config=self.tasks_config['reporting_task'],  
    )
```

```

        config=self.tasks_config['reporting_task'],

        output_file='report.md'

    )

@crew
def crew(self) -> Crew:

    """Creates the VacationPlanner crew"""

    # To learn how to add knowledge sources to your crew, check out the documentation:
    # https://docs.crewai.com/concepts/knowledge#what-is-knowledge

    return Crew(

        agents=self.agents, # Automatically created by the @agent decorator

        tasks=self.tasks, # Automatically created by the @task decorator

        process=Process.sequential,

        verbose=True,

        # process=Process.hierarchical, # In case you wanna use that instead
        https://docs.crewai.com/how-to/Hierarchical/

    )

#3 -----@agentcore.entrypoint decorator- Python decorator within the Bedrock AgentCore
SDK-----

# Function to be executed by the runtime on an event (prompt) & Creates WebServer
Endpoints

@app.entrypoint
def agent_invocation(payload, context):

    """Handler for agent invocation"""

    print(f'Payload: {payload}')

```

```
try:
```

```
    # Extract user input from payload
```

```
    user_input = payload.get("topic", "Tokyo, Japan")
```

```
    print(f"Processing vacation destination: {user_input}")
```

```
    # Crew Execution - Creates an instance of the VacationPlanner class and run crew  
method
```

```
    research_crew_instance = VacationPlanner()
```

```
    crew = research_crew_instance.crew()
```

```
    # Starts the sequential agent workflow
```

```
    result = crew.kickoff(inputs={'topic': user_input})
```

```
    print("Context:\n-----\n", context)
```

```
    print("Result Raw:\n*****\n", result.raw)
```

```
    # Safely access json_dict if it exists
```

```
    if hasattr(result, 'json_dict'):
```

```
        print("Result JSON:\n*****\n", result.json_dict)
```

```
    return {"result": result.raw}
```

```
except Exception as e:
```

```
    print(f"Exception occurred: {e}")
```

```
    return {"error": f"An error occurred: {str(e)}"}
```

```
# Local test function
```

```
def test_local():

    """Test the crew locally without AgentCore"""

    try:

        crew_instance = VacationPlanner()

        crew = crew_instance.crew()

        result = crew.kickoff(inputs={'topic': 'Plan a vacation to Germany'})

        print("Result:", result.raw)

        return result

    except Exception as e:

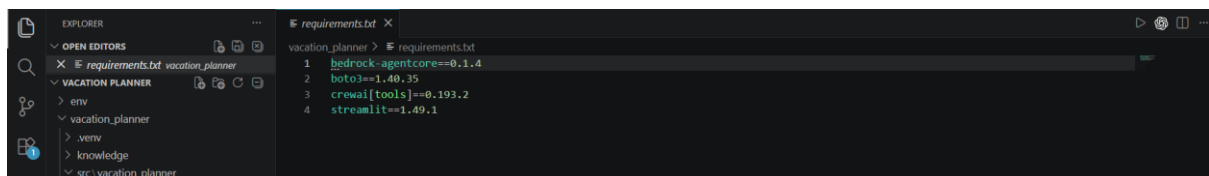
        print(f"Error: {e}")

        return None

if __name__ == "__main__":

    app.run(port=8080)
```

- Create requirements.txt
- Create:
 - requirements.txt
- Add:
 - bedrock-agentcore
 - boto3
 - crewai-tools
 - streamlit

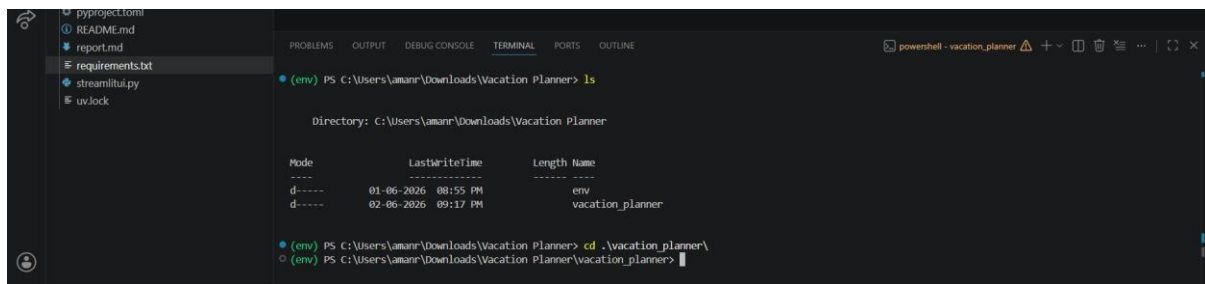


- Now we will go to our project that was created by crewai automatically
- For that run the command

```
ls
```

- then run:

```
cd <project path>
```



The screenshot shows a VS Code terminal window with the following commands and output:

```
(env) PS C:\Users\amanr\Downloads\Vacation Planner> ls
Directory: C:\Users\amanr\Downloads\Vacation Planner

Mode                LastWriteTime         Length Name
----                -
d-----          01-06-2026   08:55 PM             env
d-----          02-06-2026   09:17 PM        vacation_planner

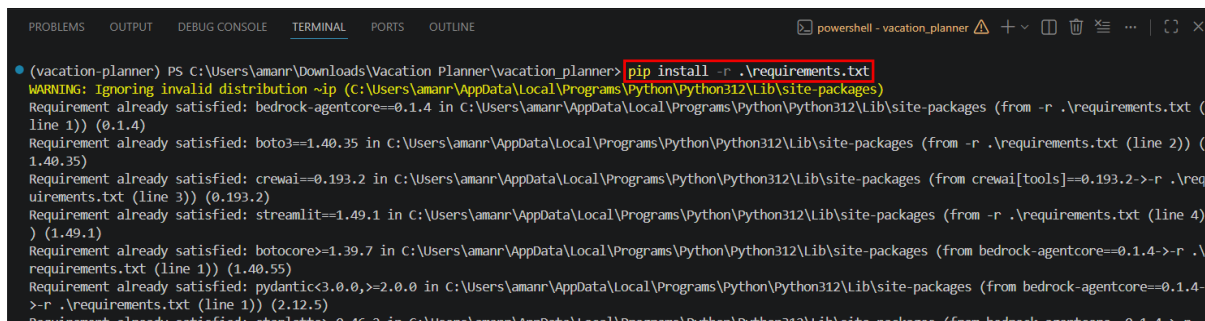
(env) PS C:\Users\amanr\Downloads\Vacation Planner> cd .\vacation_planner\
(env) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner>
```

- Now activate the virtual environment.

```
.venv\scripts\activate
```

- Then install the libraries using the commands.

```
pip install -r requirements.txt
```

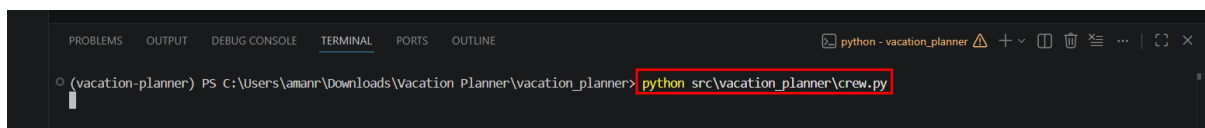


The screenshot shows a VS Code terminal window with the following output:

```
(vacation-planner) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner> pip install -r .\requirements.txt
WARNING: Ignoring invalid distribution ~ip (C:\Users\amanr\AppData\Local\Programs\Python\Python312\Lib\site-packages)
Requirement already satisfied: bedrock-agentcore==0.1.4 in C:\Users\amanr\AppData\Local\Programs\Python\Python312\Lib\site-packages (from -r .\requirements.txt (line 1)) (0.1.4)
Requirement already satisfied: boto3==1.40.35 in C:\Users\amanr\AppData\Local\Programs\Python\Python312\Lib\site-packages (from -r .\requirements.txt (line 2)) (1.40.35)
Requirement already satisfied: crewai==0.193.2 in C:\Users\amanr\AppData\Local\Programs\Python\Python312\Lib\site-packages (from crewai[tools]==0.193.2->-r .\requirements.txt (line 3)) (0.193.2)
Requirement already satisfied: streamlit==1.49.1 in C:\Users\amanr\AppData\Local\Programs\Python\Python312\Lib\site-packages (from -r .\requirements.txt (line 4)) (1.49.1)
Requirement already satisfied: botocore==1.39.7 in C:\Users\amanr\AppData\Local\Programs\Python\Python312\Lib\site-packages (from bedrock-agentcore==0.1.4->-r .\requirements.txt (line 1)) (1.40.55)
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in C:\Users\amanr\AppData\Local\Programs\Python\Python312\Lib\site-packages (from bedrock-agentcore==0.1.4->-r .\requirements.txt (line 1)) (2.12.5)
Requirement already satisfied: starlette==0.46.2 in C:\Users\amanr\AppData\Local\Programs\Python\Python312\Lib\site-packages (from bedrock-agentcore==0.1.4->-r .\requirements.txt (line 1)) (0.46.2)
```

- then run the crew.py file

```
python <path to your crew.py file>
```

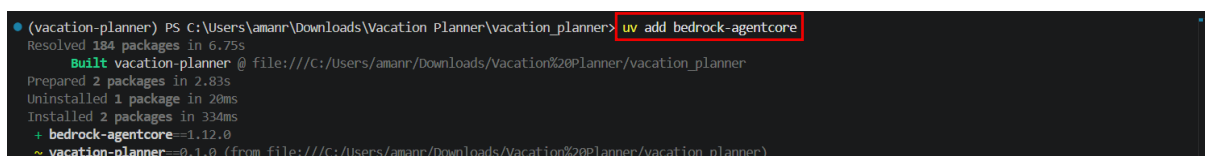


The screenshot shows a VS Code terminal window with the following command:

```
(vacation-planner) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner> python src\vacation_planner\crew.py
```

- After running the file if you get the error, no module found bedrock agentcore
- Then run the command

```
uv add bedrock-agentcore
```

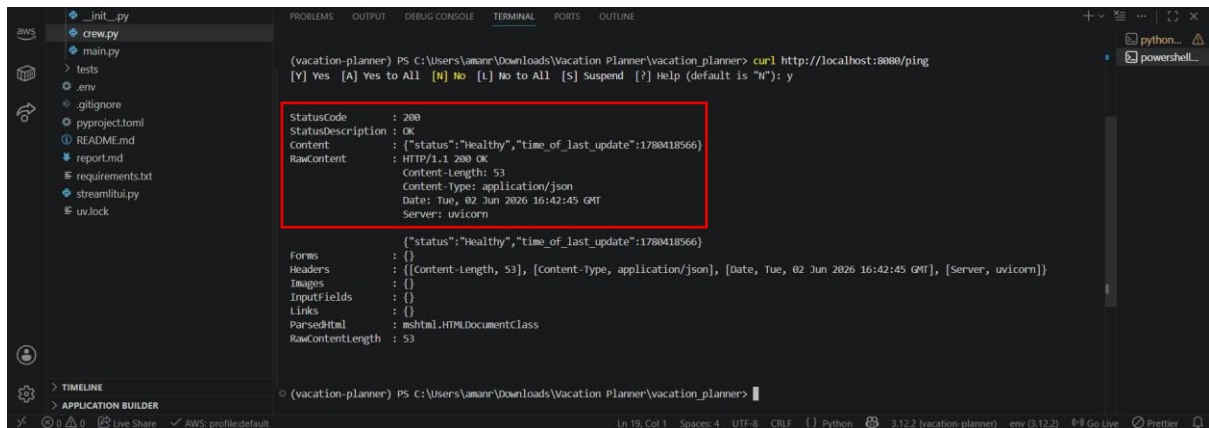


The screenshot shows a VS Code terminal window with the following output:

```
(vacation-planner) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner> uv add bedrock-agentcore
Resolved 184 packages in 6.75s
Built vacation-planner @ file:///C:/Users/amanr/Downloads/Vacation%20Planner/vacation_planner
Prepared 2 packages in 2.83s
Uninstalled 1 package in 20ms
Installed 2 packages in 334ms
+ bedrock-agentcore==1.12.0
~ vacation-planner==0.1.0 (from file:///C:/Users/amanr/Downloads/Vacation%20Planner/vacation_planner)
```

- Now execute the crew.py file again.
- Then open a new terminal and run the curl command

```
curl http://localhost:8080/ping
```



- Now we will create a docker file.
- The Dockerfile creates a lightweight Linux ARM64 container using Python 3.11, which is compatible with Amazon Bedrock AgentCore Runtime. It installs all required Python dependencies from requirements.txt, copies the CrewAI application source code into the container, and exposes port **8080** for AgentCore Runtime. When the container starts, it automatically runs the crew.py application, making the Vacation Planner agent available as a web service.

```
# Use uv package manager, ARM64 compatible architecture
```

```
FROM --platform=linux/arm64 ghcr.io/astral-sh/uv:python3.11-bookworm-slim
```

```
WORKDIR /app
```

```
# Install build tools for compiling dependencies
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
```

```
    build-essential \
```

```
    && rm -rf /var/lib/apt/lists/*
```

```
# Copy requirements.txt
```

```
COPY requirements.txt .
```

```
# Use uv pip to install dependencies
```

```
RUN --mount=type=cache,target=/root/.cache/uv \
```

```
    uv pip install --system --no-cache -r requirements.txt
```

```
# Copy app source code and pyproject.toml
```

```
COPY src/ ./src/
```


COPY pyproject.toml .

Expose port

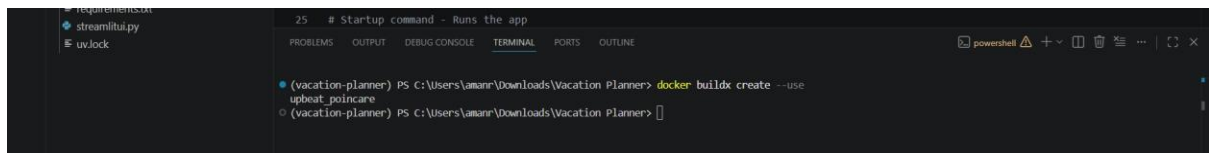
EXPOSE 8080

Startup command - Runs the app

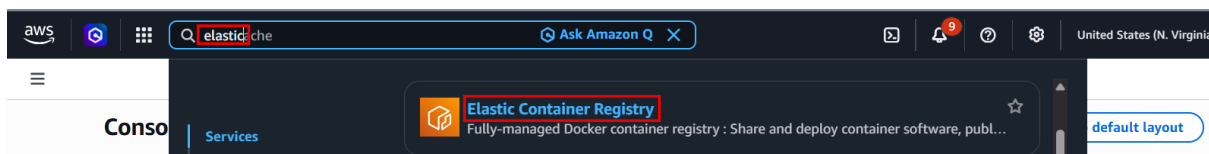
CMD ["python", "src/vacation_planner/crew.py"]

- Before running the below command run the docker on your local machine.
- Enable Docker Buildx
- Run:

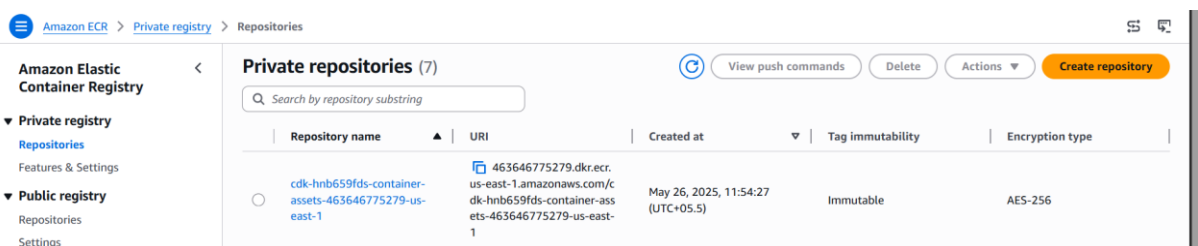
docker buildx create --use



- Create Amazon ECR Repository
- For this go to AWS console
- Navigate to ECR (Elastic Container Registry)



- We will run the command to create a repository.
- This repo will show up here.



- Then run the command

aws ecr create-repository --repository-name vacation-planner-01 --region us-east-1

- Now you can see the vacation-planner-01 here.

Amazon Elastic Container Registry					
Private registry					
Public registry					
ECR public gallery					
Amazon ECS					
Amazon EKS					
☐	javadevops1	463646775279.dkr.ecr.us-east-1.amazonaws.com/javadevops1	January 01, 2026, 11:51:38 (UTC+05.5)	Mutable	AES-256
☐	linuxapp	463646775279.dkr.ecr.us-east-1.amazonaws.com/linuxapp	January 10, 2024, 22:05:12 (UTC+05.5)	Mutable	AES-256
☐	starrepo1	463646775279.dkr.ecr.us-east-1.amazonaws.com/starrepo1	May 30, 2026, 19:50:58 (UTC+05.5)	Mutable	AES-256
☐	techcartrepo1	463646775279.dkr.ecr.us-east-1.amazonaws.com/techcartrepo1	January 09, 2026, 18:37:09 (UTC+05.5)	Mutable	AES-256
☐	vacation-planner-01	463646775279.dkr.ecr.us-east-1.amazonaws.com/vacation-planner-01	June 02, 2026, 22:53:59 (UTC+05.5)	Mutable	AES-256

- Authenticate Docker with ECR
- Run:

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 463646775279.dkr.ecr.us-east-1.amazonaws.com
```

- Replace:
- ACCOUNT_ID
- with your AWS Account ID.
- Expected output:
- Login Succeeded

- Build and Push the Container Image
- Run the build command from the folder containing the Dockerfile:

```
cd vacation_planner
```

- Build and Push
- Command structure:

```
docker buildx build --platform linux/arm64 -t 463646775279.dkr.ecr.us-east-1.amazonaws.com/vacation-planner-01:latest --push .
```

- Replace:
- ACCOUNT_ID
- Region (if needed)
- Repository name (if different)
- This will take up to 30 mins for the first time.

```
(vacation-planner) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner> docker buildx build --platform linux/arm64 -t 463646775279.dkr.ecr.us-east-1.amazonaws.com/vacation-planner-01:latest --push .
[+] Building 2.1s (0/1)
=> [internal] booting buildkit
=> => pulling image moby/buildkit:buildx-stable-1
                                docker-container:goofy_kalam 2.1s
                                2.1s
```

- Now let's check the image in our repository
- You can see the latest push.

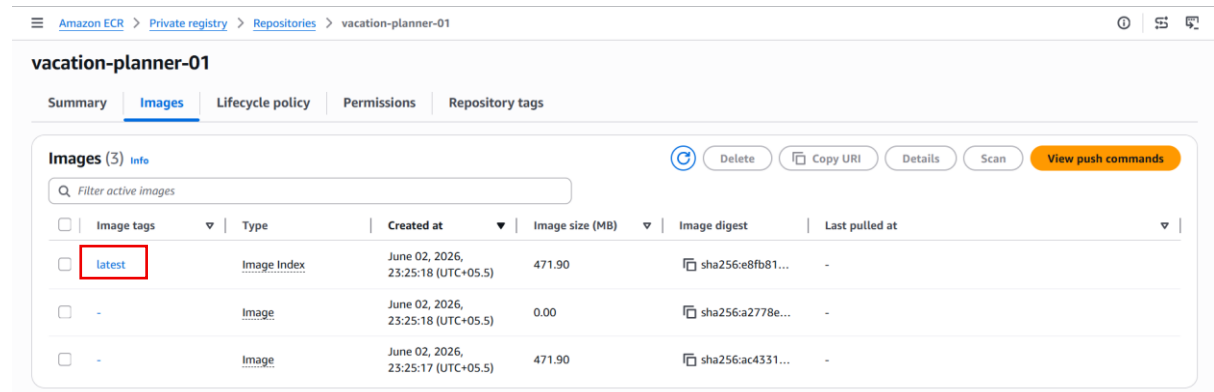
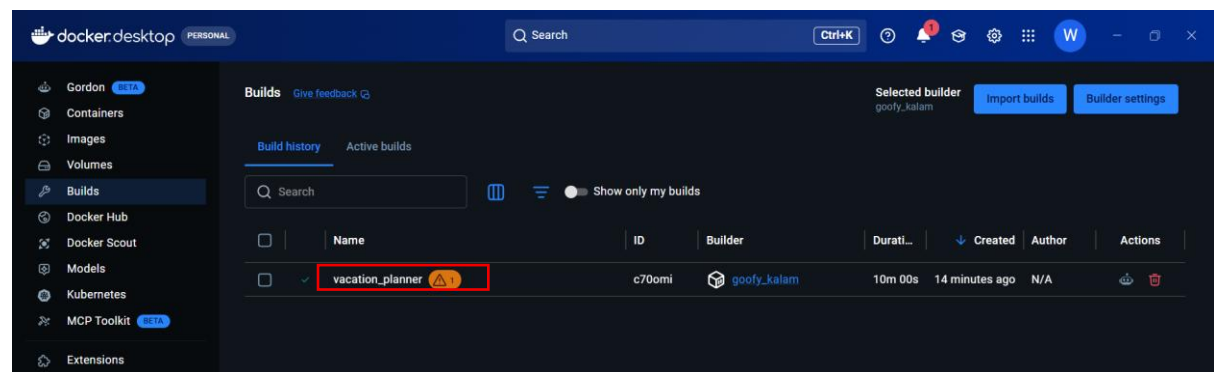


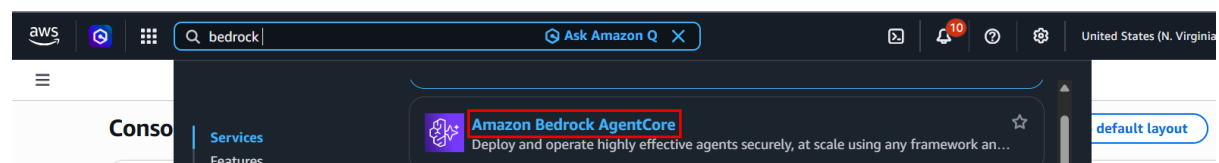
Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
latest	Image Index	June 02, 2026, 23:25:18 (UTC+05.5)	471.90	sha256:e8fb81...	-
-	Image	June 02, 2026, 23:25:18 (UTC+05.5)	0.00	sha256:a2778e...	-
-	Image	June 02, 2026, 23:25:17 (UTC+05.5)	471.90	sha256:ac4331...	-

- Also, you can see the docker build



Name	ID	Builder	Durati...	Created	Author	Actions
vacation_planner	c70omi	goofy_kalam	10m 00s	14 minutes ago	N/A	

- Let's go back to AWS Console
- Navigate to Amazon Bedrock AgentCore



- Click on Runtime and then click on Host agent/tool

Amazon Bedrock AgentCore > Runtime

Amazon Bedrock AgentCore

- Discover
 - Overview
 - Registry [Preview](#)
- Build
 - Harness [Preview](#)
 - Runtime**
 - Gateways
 - Memory
 - Policy [New](#)
 - Identity
 - Payments [Preview](#)
- Built-in tools
 - Browser
 - Code Interpreter
- Test
 - Harness playground
 - Runtime playground

Build an agent or tool

Create your agent or tool by either starting with a template and uploading to Amazon S3, or by building a Docker image using any framework and uploading to Amazon ECR.

Host in Runtime

Host your agents and tools using [runtime](#) by pointing to your code or container, and get a secure, serverless, and scalable endpoint to serve your sessions in an isolated fashion.

[Host agent/tool](#)

Assess performance using observability

Monitor key metrics, sessions, traces and spans to get real-time insights on performance, accuracy, and reliability of your agent.

Learn more about enabling observability for a [specific resource](#) or at the [account level](#).

[View observability](#)

Runtime resources (0) [Info](#)

Name	Status	Managed by	Description	Latest version	Last updated (U...
No agents found					

[Delete](#) [Edit](#) **Host agent/tool**

- Configure Agent Details
- Agent Name
- Example:
- vacation_planner_agent
- Description
- Example:
- Provides detailed vacation plans for tourist locations

Amazon Bedrock AgentCore > Runtime > Host Agent

Host agent or tool [Info](#)

This workflow provides a streamlined process for deploying externally-developed agents or tools on Amazon Bedrock AgentCore with enterprise-grade security and scalability.

Agent or Tool details

Name

Valid characters are a-z, A-Z, 0-9 and _ (underscore). The name can have up to 48 characters.

Additional details

Description - optional

Maximum 4096 characters

- Select Docker Image from ECR
- Choose:
- Import Docker Image from ECR
- Click:
- Browse Images

Agent Source

Source type

☐ S3 Source
Upload or choose an agent directly from an S3 bucket.

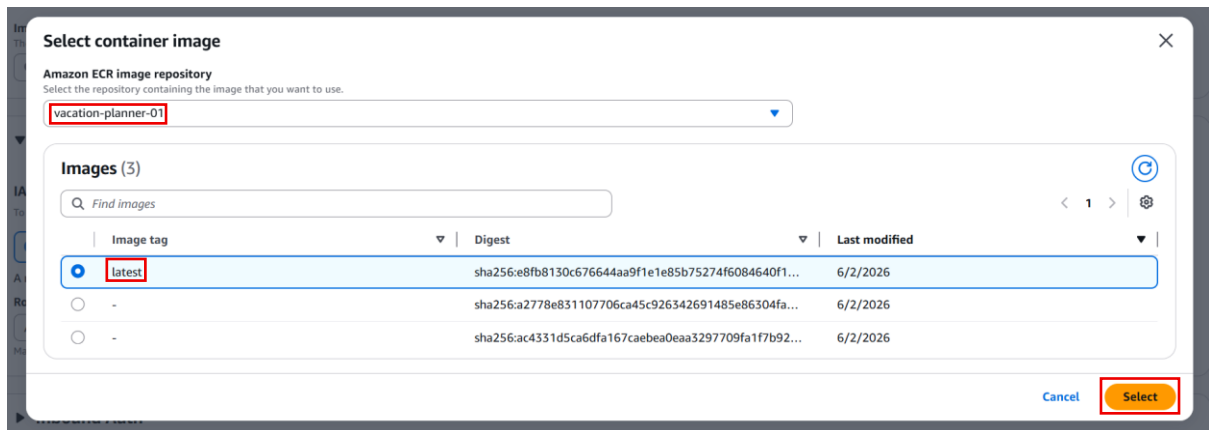
☒ **ECR Container**
Deploy your agent using an Amazon ECR container image.

Image URI

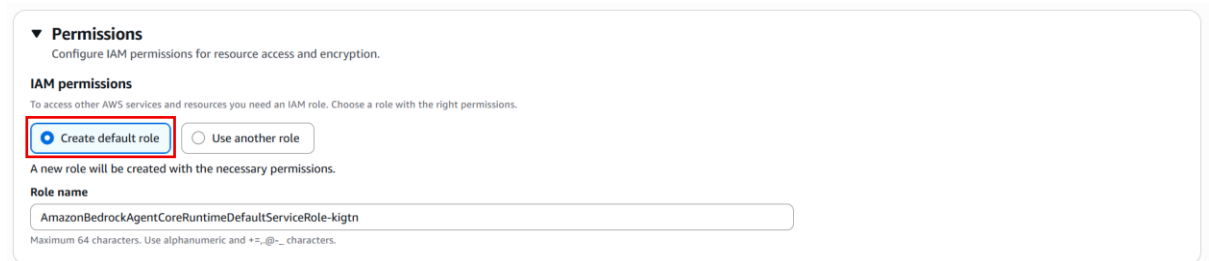
The location of the container image to use for your agent.

[View](#) [Browse images](#)

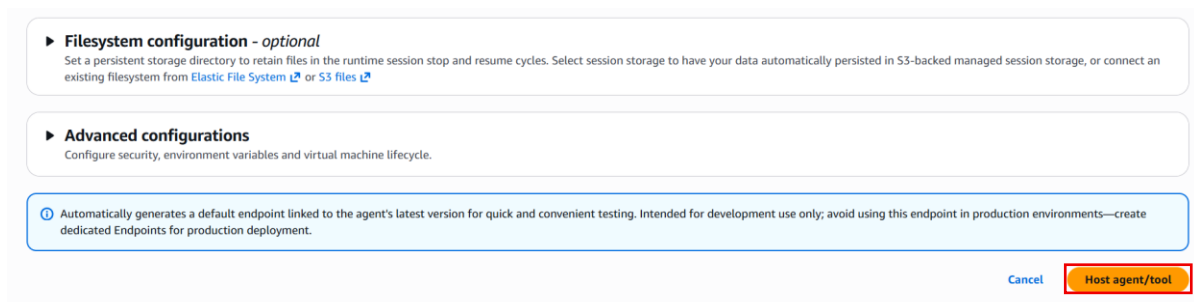
- Select:
- vacation_planner_01
- Repository created in the previous lab.
- Choose image tag:
- latest
- Click:
- Select



- Under Runtime Permissions:
- Create and use a new service role
- Bedrock AgentCore automatically creates the IAM role.



- Click Host agent/tool



- Now our agent is ready and created successfully.

✓ Vacation_planner_agent, Version 1 was successfully created. A default endpoint is also in progress and will be associated with version 1. Update your IAM execution role with all resource IDs that are part of your agent code to test your default endpoint. [Update IAM execution role](#) ✕

Vacation_planner_agent

[Delete](#) [Update hosting](#)

▼ Agent and tool details

Name
Vacation_planner_agent

Description
Provides detailed vacation plans for tourist locations

Runtime ARN
[arn:aws:bedrock-agentcore:us-east-1:463646775279:runtime/Vacation_planner_agent-m950lpBDcX](#)

Last updated date
June 03, 2026, 22:36 (UTC+05:30)

Created date
June 03, 2026, 22:36 (UTC+05:30)

Status
Ready

Source type
ECR

Runtime ID
Vacation_planner_agent-m950lpBDcX

- Now we will give the admin access to the created IAM role
- Click on Version 1

Versions (1) [Info](#)

A snapshot of the agent created automatically with each update, allowing you to track changes, manage rollbacks, and reference specific states.

< 1 > ⚙️

Version	Status	Description	Created at (UTC+05:30)
Version 1	Ready	Provides detailed vacation plans fo...	June 03, 2026, 22:36

- Then Click on the IAM role.

Permissions

IAM service role

[AmazonBedrockAgentCoreRuntimeDefaultServiceRole-kigtn](#)

- Now click on the add permission to add the permission
- Then click on attach policies

[Permissions](#) | [Trust relationships](#) | [Tags](#) | [Last Accessed](#) | [Revoke sessions](#)

Permissions policies (1) [Info](#)

You can attach up to 10 managed policies.

Filter by Type All types

[Add permissions](#) [Attach policies](#) [Create inline policy](#)

☐	Policy name	Type	Attached entities
☐	AmazonBedrockAgentCoreRuntimeExecutionPolicy_bc5...	Customer managed	1

[Permissions](#) | [Trust relationships](#) | [Tags](#) | [Last Accessed](#) | [Revoke sessions](#)

Permissions policies (2) [Info](#)

You can attach up to 10 managed policies.

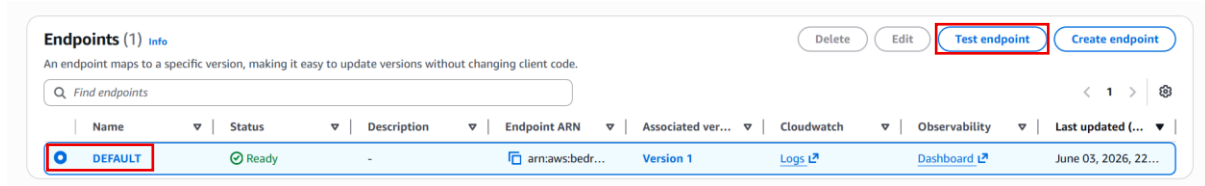
Filter by Type All types

[Add permissions](#)

☐	Policy name	Type	Attached entities
☐	AdministratorAccess	AWS managed - job function	34
☐	AmazonBedrockAgentCoreRuntimeExecutionPolicy_bc5...	Customer managed	1

- Now we will test our agent through the endpoint

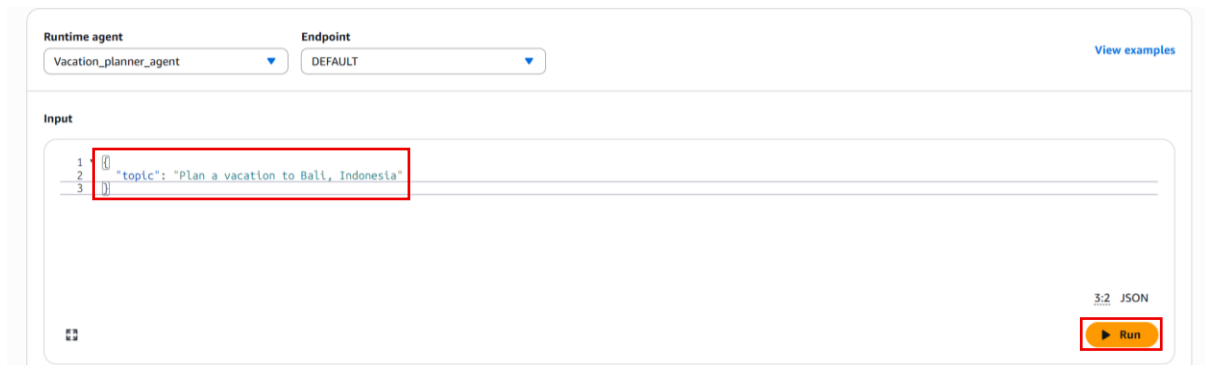
- Select the default endpoint.
- And then click test endpoint



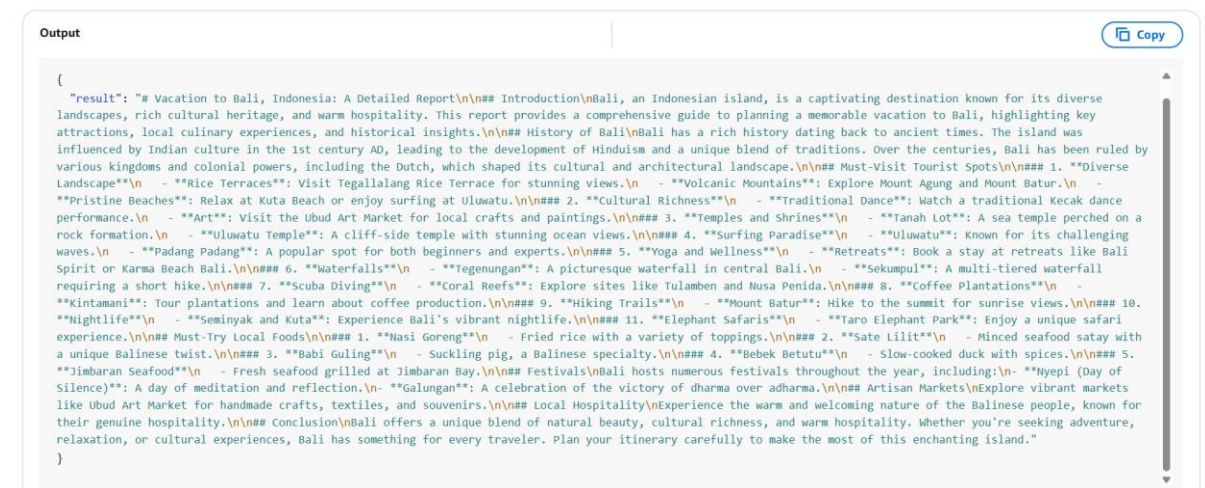
- Give the prompt

```
{
  "topic": "Plan a vacation to Bali, Indonesia"
}
```

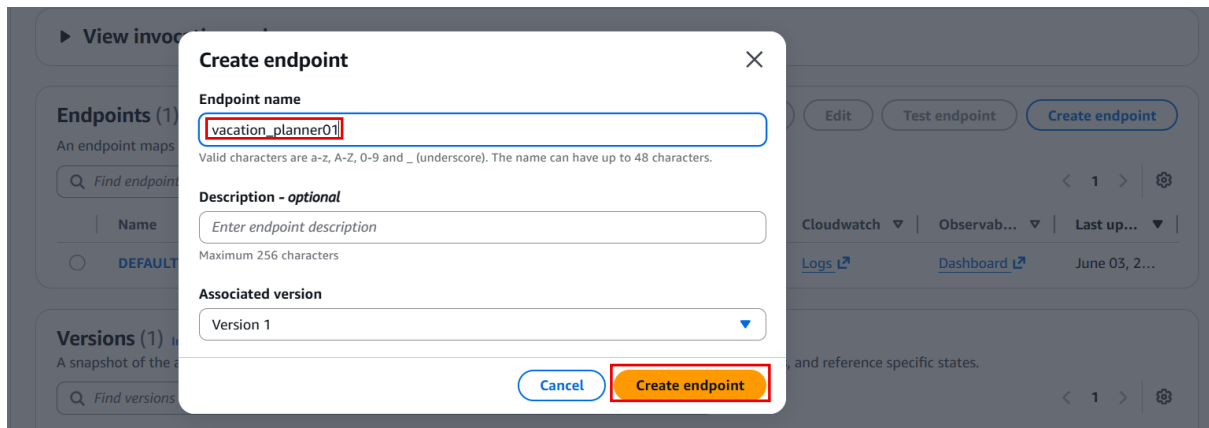
- Click run



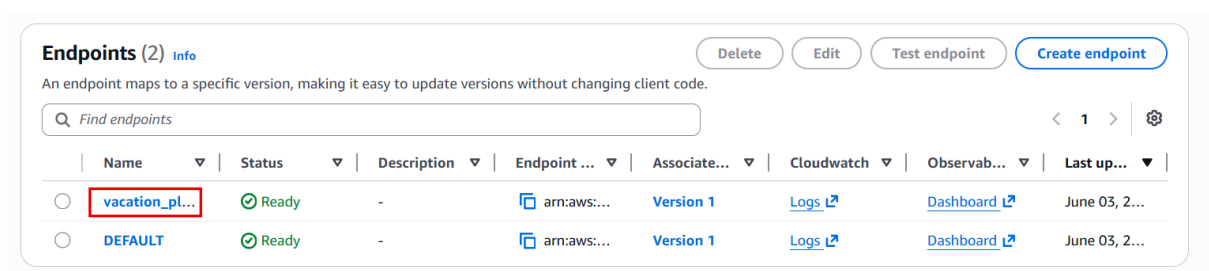
- Now you can see the output



- Now go to the endpoint and create a new endpoint
- Click create endpoint



- We will use this endpoint



- In view invocation code, there is code template and we will use these code template in lambda function.



- Create a lambda function
- Click Create function.
- Give the name and select the runtime

[Lambda](#) > [Functions](#) > Create function

Create function Info

Choose one of the following options to create your function.

☒ **Author from scratch**
 Start with a simple Hello World example.

☐ **Use a blueprint**
 Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
 Select a container image to deploy for your function.

Basic information

Function name
 Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime Info
 Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Permissions
 By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution role** under **Additional settings**.

- In configuration, increase the timeout

[Lambda](#) > [Functions](#) > [Vacation-planner01](#) > Edit basic settings

Your function is allocated CPU proportional to the memory configured.

MB
Set memory to between 128 MB and 10240 MB.

Ephemeral storage Info
 You can configure up to 10 GB of ephemeral storage (/tmp) for your function. [View pricing](#)
 MB
Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.

SnapStart Info
 Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the [SnapStart compatibility considerations](#). For Python and .NET runtimes, [view pricing](#).

Supported runtimes: .NET 10 (C#/F#/PowerShell), .NET 8 (C#/F#/PowerShell), Java 11, Java 17, Java 21, Java 25, Python 3.12, Python 3.13, Python 3.14.

Timeout
 min sec

Execution role
 To access other AWS services and resources your function needs an IAM role. Choose a role with the right permissions for your function.
 [Create new role](#)
[View role details in IAM](#)

[Cancel](#) [Save](#)

- Now we have to provide the permissions.
- Go to the permissions
- Click on the execution role

[Code](#) | [Test](#) | [Monitor](#) | [Configuration](#) | [Aliases](#) | [Versions](#)

Configuration

General configuration

Triggers

[Permissions](#)

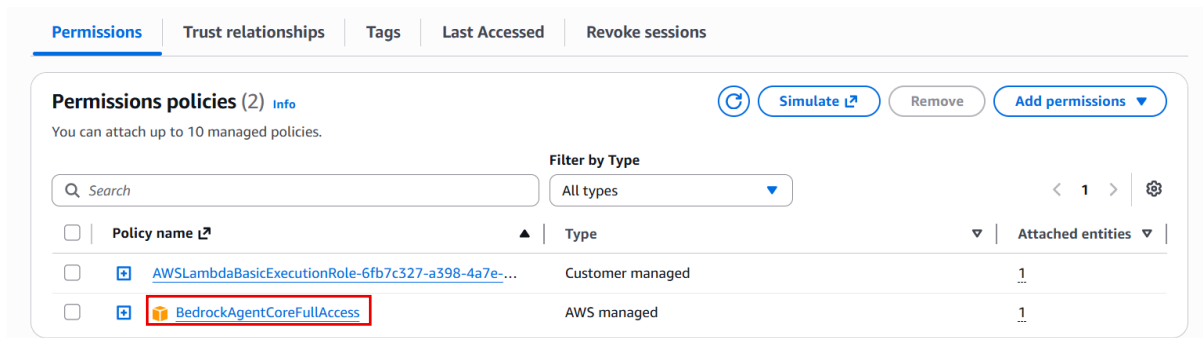
Execution role

Role name

[View role document](#) [Edit](#)

Resource summary

- Attach this policy



- Let's go back to the lambda function and the go to code
- The Lambda function initializes the required libraries and creates a Bedrock AgentCore client to establish communication with the deployed Vacation Planner agent. It then extracts the user's prompt from the incoming request, converts it into the agent's expected payload format, and generates a unique session ID to maintain a separate runtime session.
- Using the generated payload and session ID, the function invokes the AgentCore runtime and receives the agent's response. Finally, it parses the returned data and sends a successful HTTP response containing the agent output, session details, and CORS headers for web application access.

#1 Imports

```
import json
```

```
import boto3
```

```
import uuid
```

```
def lambda_handler(event, context):
```

```
    #2 Create a client connection with Bedrock AgentCore
```

```
    client = boto3.client('bedrock-agentcore', region_name='us-east-1')
```

```
    #3 Get user input from event, match to the expected Agent payload structure
```

```
    user_input = event.get('prompt', 'Tokyo')
```

```
    payload = json.dumps({"topic": user_input})
```

```
    #4 Generate unique session ID (must be 33+ characters) using UUID without hyphens
```

```

session_id = f"lambda_session_{str(uuid.uuid4()).replace('-', '')}"

print(f"Invoking AgentCore with payload: {payload} & session_id: {session_id}")

#5 Invoke the Agentcore runtime with the vacation planner agent
response = client.invoke_agent_runtime(

    agentRuntimeArn='arn:aws:bedrock-agentcore:us-east-
1:463646775279:runtime/Vacation_planner_agent-m9SOipBDcX',

    runtimeSessionId=session_id, # Must be 33+ char. Every new SessionId will create a new
MicroVM

    payload=payload,

    qualifier="vacation_planner01" # This is Optional. When the field is not provided,
Runtime will use DEFAULT endpoint
)

#6 read and parse the response from the agentcore
response_body = response['response'].read()
response_data = json.loads(response_body)
print("Agent Response:", response_data)

#7 Return the successful response with the CORS headers
return {

    'statusCode': 200,

    'headers': {

        'Content-Type': 'application/json',

        'Access-Control-Allow-Origin': '*' # Allow requests from any origin

    },

    'body': json.dumps({

```

```
'message': 'Agent invoked successfully',

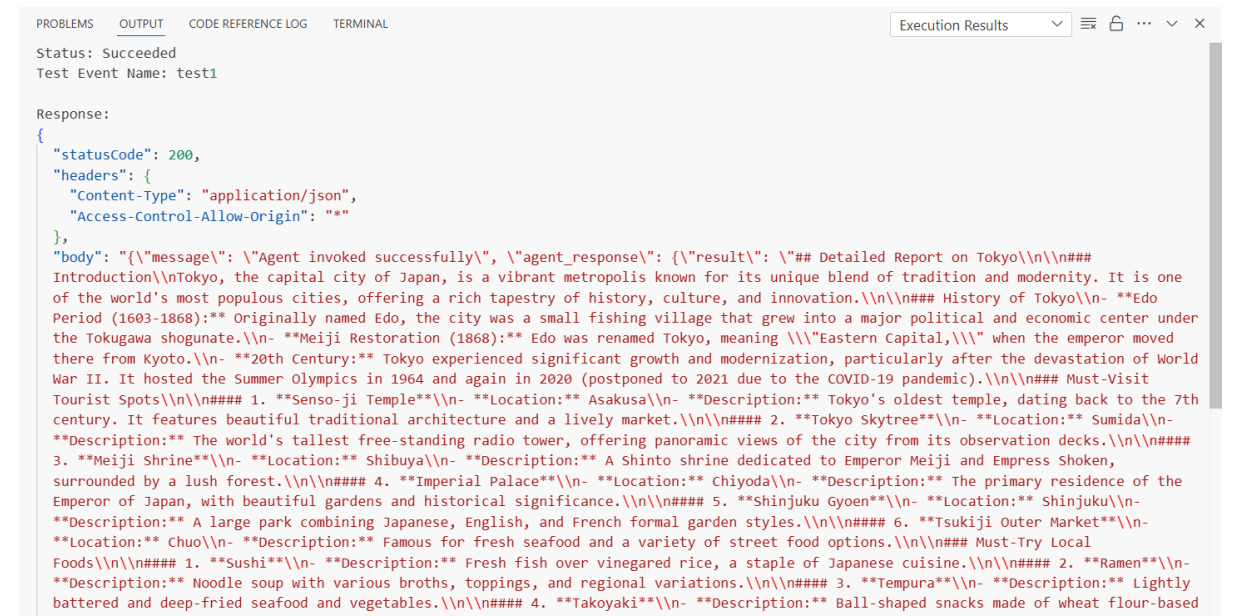
'agent_response': response_data,

'session_id': session_id

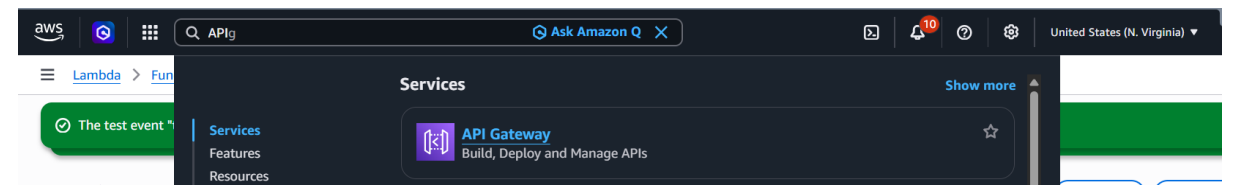
}))

}
```

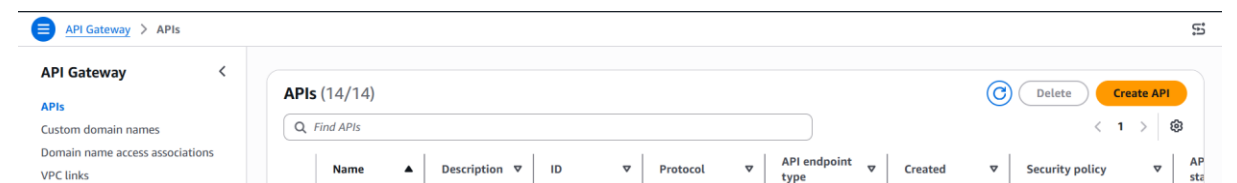
- Now we will create a test event
- ```
{
 "prompt": "Tokyo"
}
```
- Now will run the test event and wait for 15 seconds



- Now we will create an API Gateway
- Navigate to API Gateway on AWS Console



- Click Create API



- Click on the Rest API
- Click Build

### REST API

Develop a REST API where you gain complete control over the request and response along with API management capabilities.

**Works with the following:**  
Lambda, HTTP, AWS Services

Import
Build

- API Name
- Example:
- VacationPlanner01
- Click Create API

**API name**

**Description - optional**

**API endpoint type**  
Regional APIs are deployed in the current AWS Region. Edge-optimized APIs route requests to the nearest CloudFront Point of Presence. Private APIs are only accessible from VPCs.

Regional

**Security policy - new** [Info](#)  
Transport Layer Security (TLS) protects data in transit between a client and server. The security policy also determines the cipher suite options that clients can use with your API.

Choose a security policy

**IP address type** [Info](#)  
Select the type of IP addresses that can invoke the default endpoint for your API.

☒ IPv4  
Supports only edge-optimized and Regional API endpoint types.

☐ Dualstack  
Supports all API endpoint types.

Cancel
Create API

- Create a Resource
- A resource represents the URL path.
- Click:
- Create Resource

### Resources

API actions
Deploy API

Create resource

|   |                                                                       |
|---|-----------------------------------------------------------------------|
| / | <b>Resource details</b> <div> Update documentation Enable CORS </div> |
|   | Path<br>/ Resource ID<br>4saho94coi                                   |

- Resource Name:
- vacationplanner01
- Enable CORS
- Enable:
- Cross-Origin Resource Sharing (CORS)

API Gateway > APIs > Resources - VacationPlanner01 (3x5wus0axd) > Create resource

### Create resource

**Resource details**

☐ Proxy resource [info](#)  
Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

**Resource path**  
/

**Resource name**  
vacationplanner01

☒ CORS (Cross Origin Resource Sharing) [info](#)  
Create an OPTIONS method that allows all origins, all methods, and several common headers.

[Cancel](#) [Create resource](#)

- Create a POST Method
- Select the resource:  
/vacationplanner01
- Click:
- Create Method
- Choose:
- POST
- Reason:
- Bedrock AgentCore expects requests through HTTP POST.

API Gateway > APIs > Resources - VacationPlanner01 (3x5wus0axd) > Create method

### Create method

**Method details**

**Method type**  
POST

**Integration type**

☒ **Lambda function**  
Integrate your API with a Lambda function.

☐ HTTP  
Integrate with an existing HTTP endpoint.

☐ Mock  
Generate a response based on API Gateway mappings and transformations.

☐ AWS service  
Integrate with an AWS Service.

☐ VPC link  
Integrate with a resource that isn't accessible over the public internet.

- Then Select the lambda function

**Lambda function**  
Provide the Lambda function name or alias. You can also provide an ARN from another account.

us-east-1

arn:aws:lambda:us-east-1:463646775279:function:Vacation-planner01

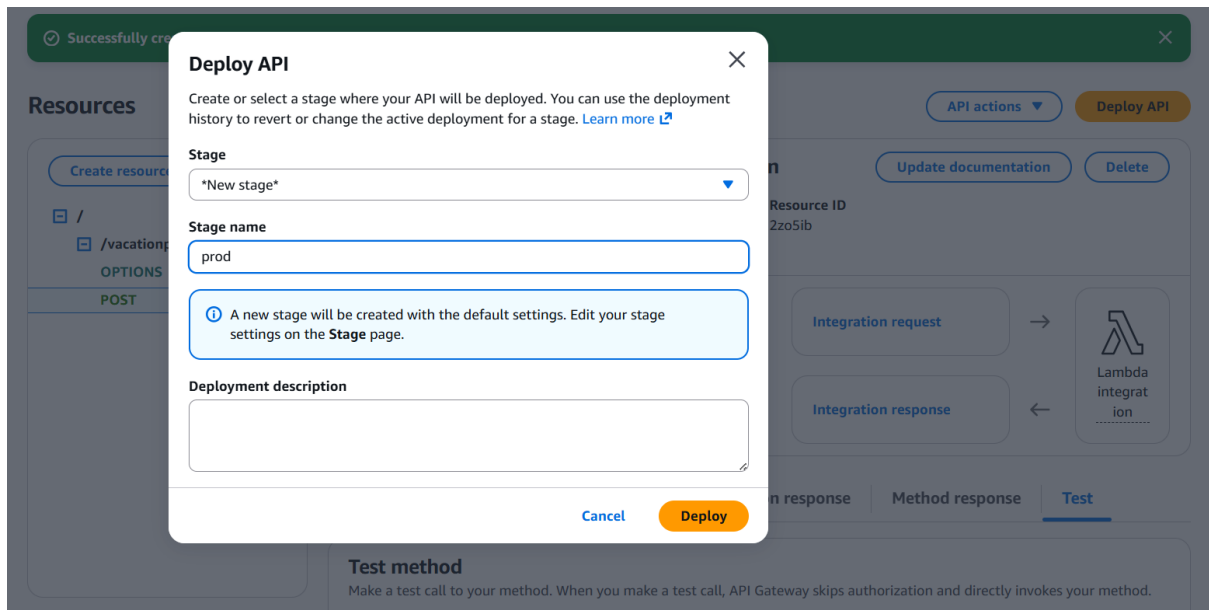
**Grant API Gateway permission**  
When you save your changes, you must grant the API Gateway permission to invoke this function.

**Integration timeout** [info](#)  
By default, you can enter an integration timeout in milliseconds and cannot be increased further.

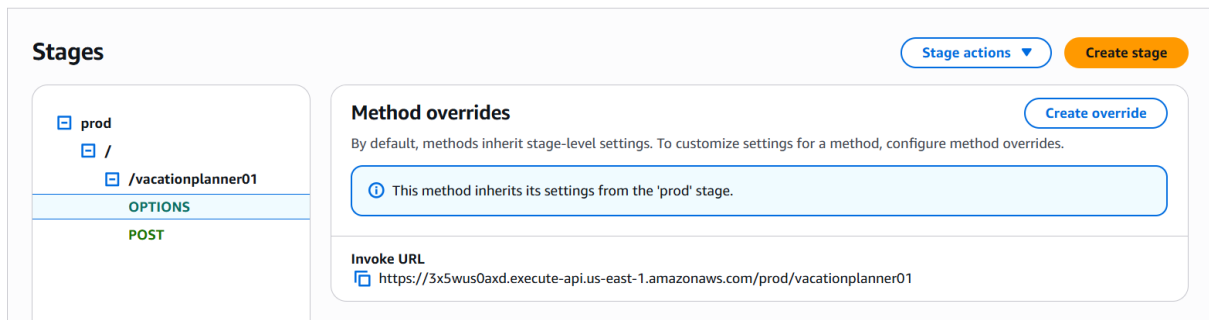
29000

arn:aws:lambda:us-east-1:463646775279:function:Vacation-planner01

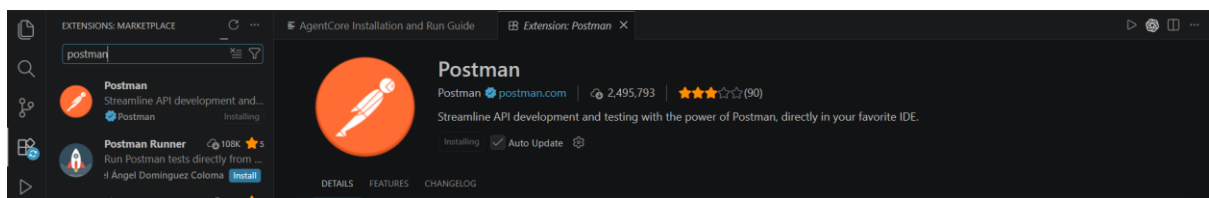
- Click create method
- Now click on Deploy API
- Click New stage



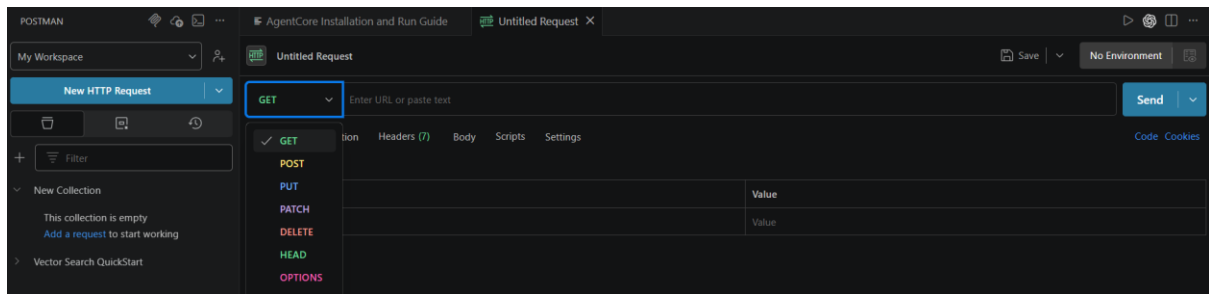
- Obtain the Invoke URL
- After deployment:
- Select the POST method.
- Copy the:
- Invoke URL



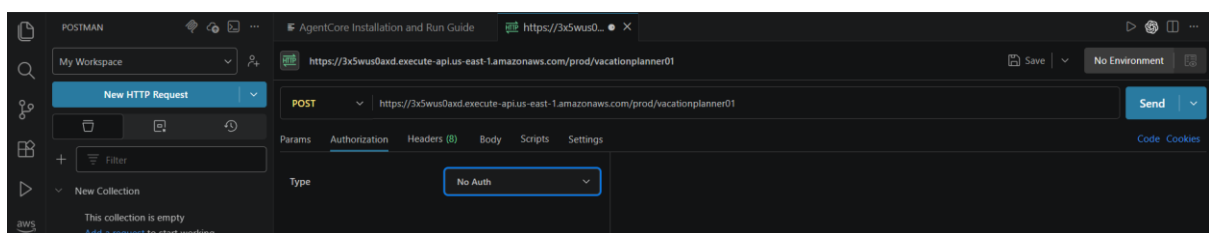
- Now go back to VSCode
- Install the extension POSTMAN



- Sign In to the Postman
- After sign in, click on the New HTTP Request
- Select the post method
- Copy the URL and paste in the Postman
- Click Send

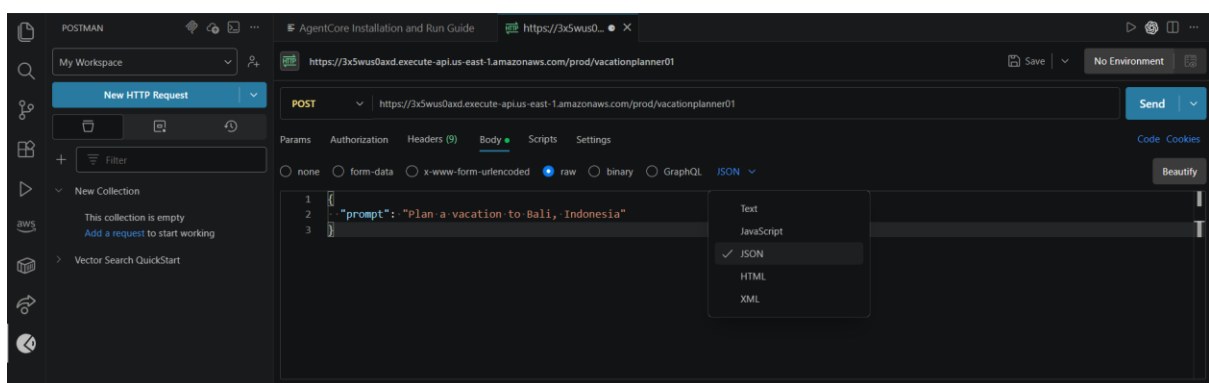


- Configure Authorization
- Select:
- No Auth
- No authentication was configured in this lab.



- Configure Request Body
- Select:
- Body → Raw → JSON
- Provide input:

```
{
 "prompt": "Plan a vacation to Bali, Indonesia"
}
```



- So, now we got our response and it took about 15 seconds







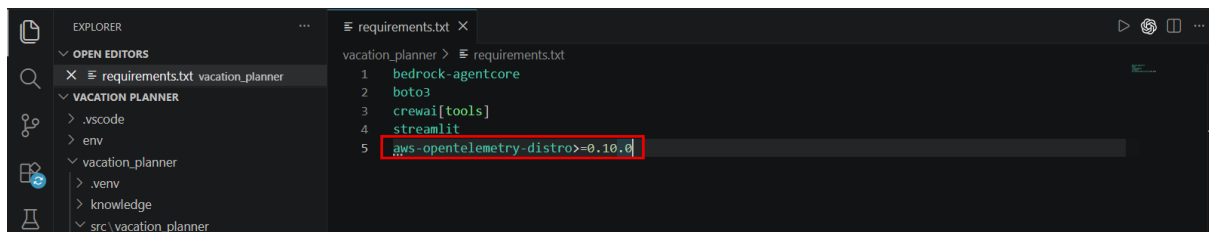
# Bedrock Agentcore Observability

## To Begin with the Lab

### Summary of the Lab

This implementation adds end-to-end observability for the Bedrock AgentCore application by integrating the AWS OpenTelemetry Distribution (ADOT) into the containerized workload. The Docker image is updated to install OpenTelemetry dependencies, configure telemetry exporters, sampling, and service metadata, and automatically instrument the application at startup without code changes. CloudWatch Transaction Search is enabled to support distributed tracing, while Bedrock Model Invocation Logging is configured to stream model activity to CloudWatch Logs. The updated application is deployed through Amazon ECR and AgentCore, enabling centralized monitoring of traces, logs, latency, token usage, request volume, and model performance for operational visibility and troubleshooting.

- Add the AWS OpenTelemetry Distribution (ADOT) dependency (aws-opentelemetry-distro==0.10.0) to requirements.txt to enable metrics, traces, and log collection.

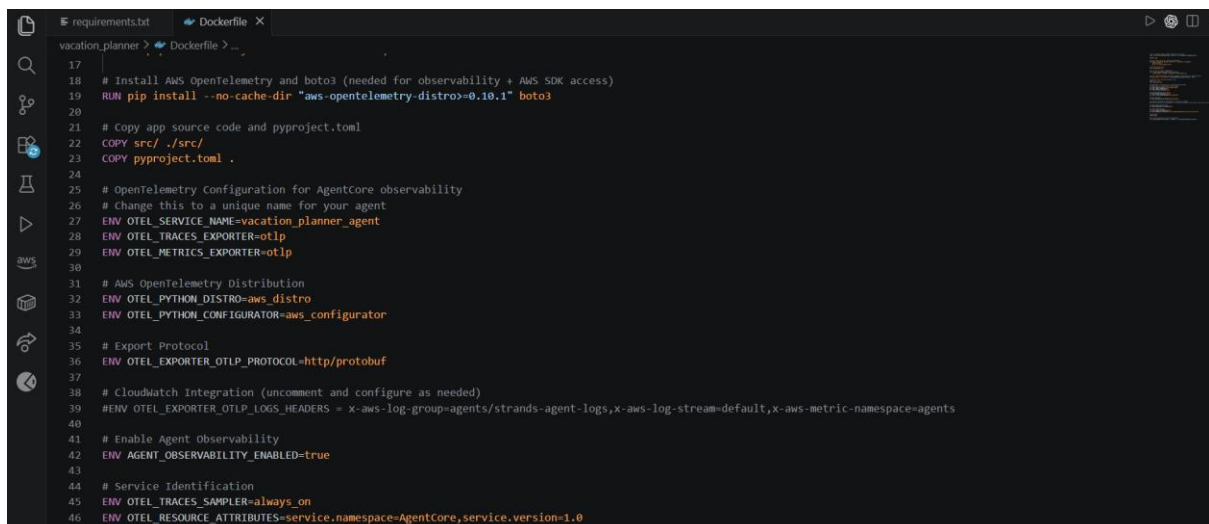


A screenshot of a code editor interface. The left sidebar shows a file explorer with a tree view containing 'OPEN EDITORS', 'requirements.txt', 'vacation\_planner', and 'VACATION PLANNER'. The main editor window displays the 'requirements.txt' file with the following content: 

```
1 bedrock-agentcore
2 boto3
3 crewai[tools]
4 streamlit
5 aws-opentelemetry-distro==0.10.0
```

 The line 'aws-opentelemetry-distro==0.10.0' is highlighted with a red box.

- Update the Dockerfile to install dependencies from requirements.txt and configure OpenTelemetry environment variables, including service name, OTLP exporters, AWS distribution settings, sampling, and observability options.



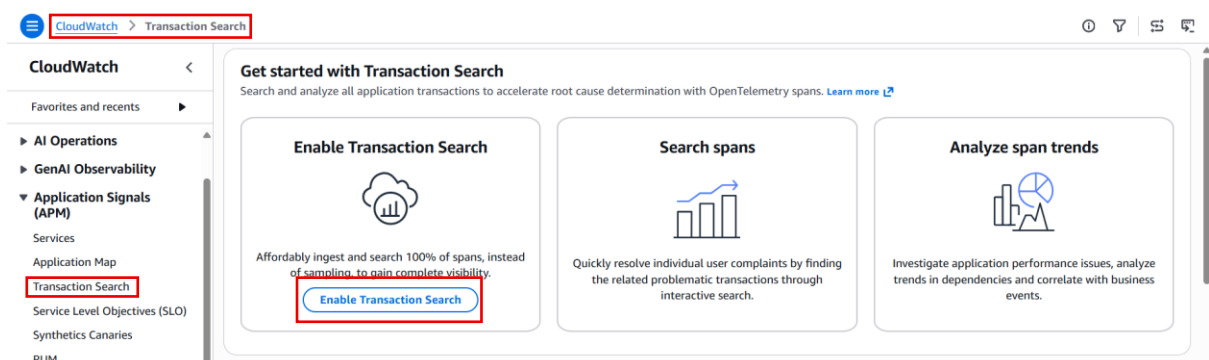
A screenshot of a code editor interface showing the 'Dockerfile' file. The left sidebar shows a file explorer with a tree view containing 'requirements.txt', 'Dockerfile', and 'vacation\_planner'. The main editor window displays the 'Dockerfile' file with the following content: 

```
17
18 # Install AWS OpenTelemetry and boto3 (needed for observability + AWS SDK access)
19 RUN pip install --no-cache-dir "aws-opentelemetry-distro==0.10.1" boto3
20
21 # Copy app source code and pyproject.toml
22 COPY src/ ./src/
23 COPY pyproject.toml .
24
25 # OpenTelemetry Configuration for AgentCore observability
26 # Change this to a unique name for your agent
27 ENV OTEL_SERVICE_NAME=vacation_planner_agent
28 ENV OTEL_TRACES_EXPORTER=otlp
29 ENV OTEL_METRICS_EXPORTER=otlp
30
31 # AWS OpenTelemetry Distribution
32 ENV OTEL_PYTHON_DISTRO=aws_distro
33 ENV OTEL_PYTHON_CONFIGURATOR=aws_configurator
34
35 # Export Protocol
36 ENV OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf
37
38 # Cloudwatch Integration (uncomment and configure as needed)
39 #ENV OTEL_EXPORTER_OTLP_LOGS_HEADERS = x-aws-log-group=agents/strands-agent-logs,x-aws-log-stream=default,x-aws-metric-namespace=agents
40
41 # Enable Agent Observability
42 ENV AGENT_OBSERVABILITY_ENABLED=true
43
44 # Service Identification
45 ENV OTEL_TRACES_SAMPLER=always_on
46 ENV OTEL_RESOURCE_ATTRIBUTES=service.namespace=AgentCore,service.version=1.0
47
```

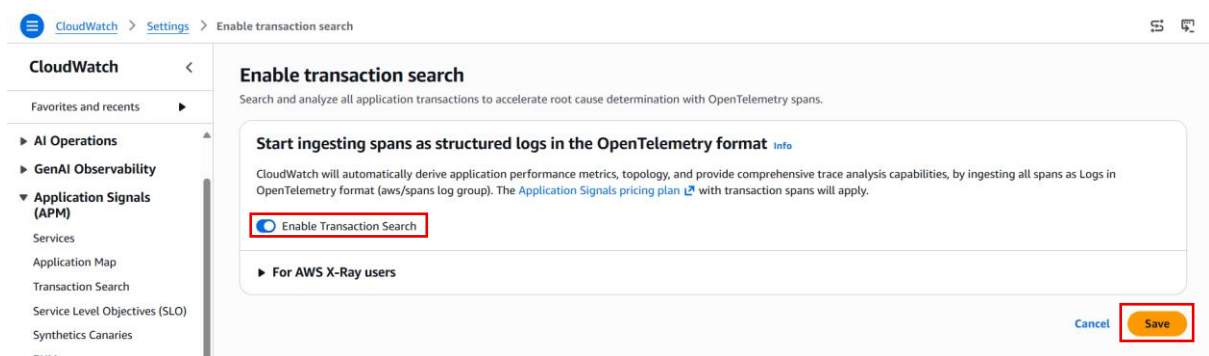
- Finally, replace the standard startup command with opentelemetry-instrument python src/vacation\_planner\_crew.py to automatically instrument the application and capture telemetry without requiring code changes.

```
47
48 # Expose port
49 EXPOSE 8080
50
51 # Run with OpenTelemetry auto-instrumentation
52 CMD ["opentelemetry-instrument", "python", "src/vacation_planner/crew.py"]
```

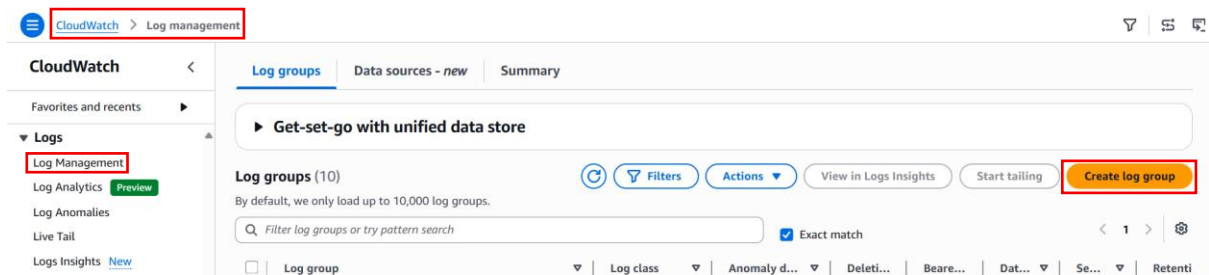
- Now go back to AWS console
- Enable CloudWatch Transaction Search
- Open:
  - CloudWatch
- Navigate to:
  - Application Signals
  - Transaction Search
- Click:
  - Enable Transaction Search



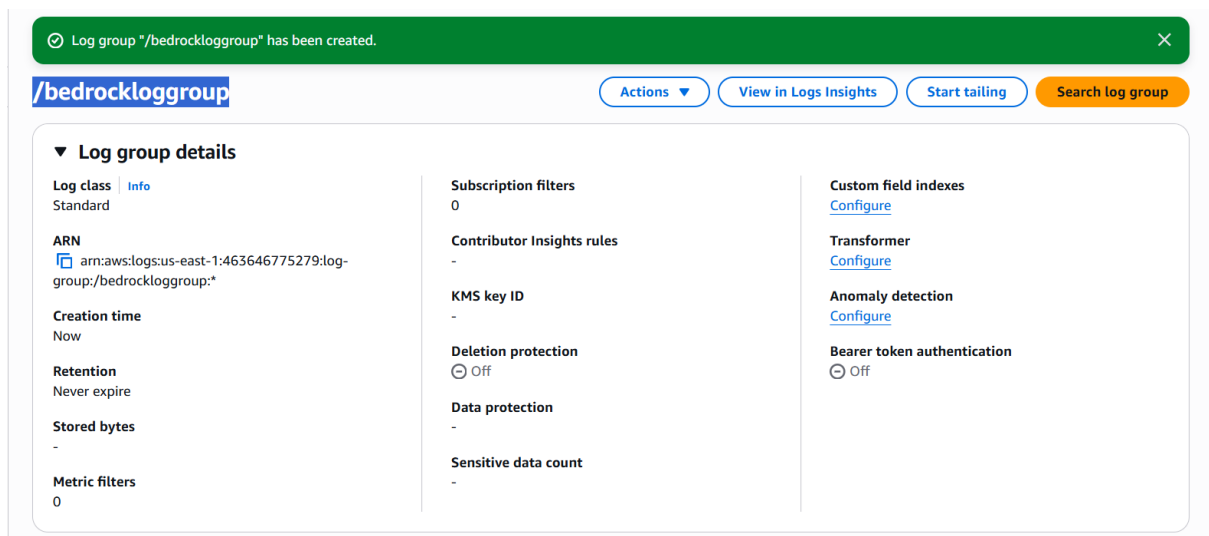
- Save changes.
- Purpose:
  - Enables distributed tracing
  - Allows transaction-level observability



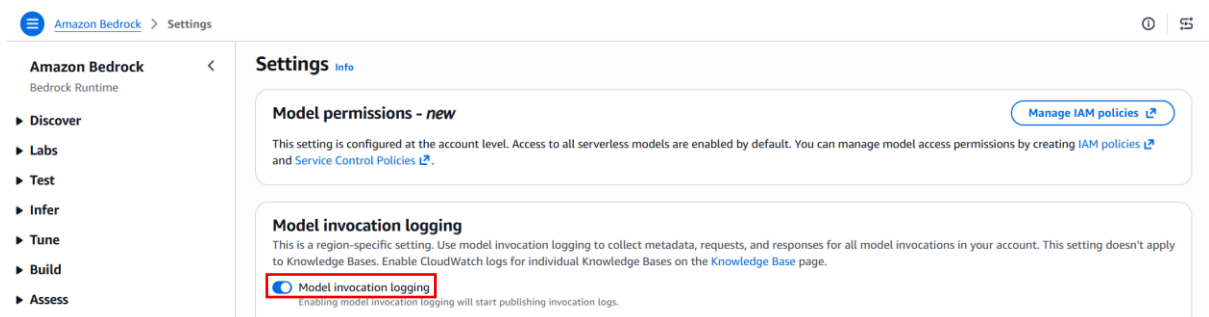
- Create CloudWatch Log Group
- Open:
  - CloudWatch
- Navigate to:
  - Log Management
- Click:
  - Create Log Group



- Example:
  - /bedrockloggroup
- Create the log group.



- Enable Bedrock Model Invocation Logging
- Open:
  - Amazon Bedrock
- Navigate to:
  - Settings
- Select:
  - Model Invocation Logging



- Enable Logging For
- Select:
  - Text
  - Image

- Embedding
- Video
- (Or only the types you need.)
- Configure Log Destination
- Choose:
  - CloudWatch Logs

- Configure:
- Log Group Name:
  - /bedrockloggroup
- Allow Bedrock to create the service role automatically.
- Click:
  - Save Settings

- Now let's build a new image with the updated docker file
- Go to the directory.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS OUTLINE
powershell - vacation_planner
(vacation-planner) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner>

```

- Note: Open Docker Desktop before running the command
- Open the terminal and run the command
- Run:

docker buildx create --use

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS OUTLINE
powershell - vacation_planner

(vacation-planner) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner> docker buildx create --use
epic_goldstine
(vacation-planner) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner>
```

- Now let's create a New ECR Repository
- Run the command

```
aws ecr create-repository --repository-name vacation-planner-observability --region us-east-1
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS OUTLINE
powershell - vacation_planner

(vacation-planner) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner> aws ecr create-repository --repository-name vacation-planner-observability --region us-east-1
{
 "repository": {
 "repositoryArn": "arn:aws:ecr:us-east-1:463646775279:repository/vacation-planner-observability",
 "registryId": "463646775279",
 "repositoryName": "vacation-planner-observability",
 "repositoryUri": "463646775279.dkr.ecr.us-east-1.amazonaws.com/vacation-planner-observability",
 "createdAt": "2026-06-07T00:23:33.000Z",
 "imageTagMutability": "IMMUTABLE",
 "imageScanningConfiguration": {
 "scanOnPush": false
 },
 "encryptionConfiguration": {
 "encryptionType": "AES256"
 }
 }
}
```

|                                                                                                                    |                                                                                                                                                                |                                    |         |         |
|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|---------|---------|
|  vacation-planner-observability |  463646775279.dkr.ecr.us-east-1.amazonaws.com/vacation-planner-observability | June 07, 2026, 00:23:33 (UTC+05.5) | Mutable | AES-256 |
|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|---------|---------|

- Now let's login to the ECR

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 463646775279.dkr.ecr.us-east-1.amazonaws.com
```

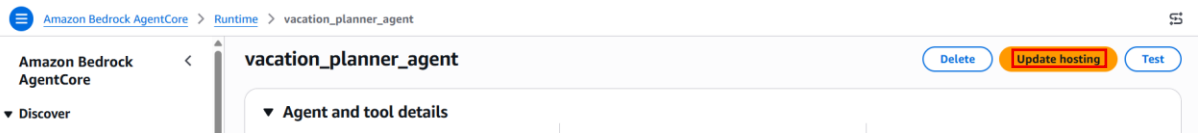
```
(vacation-planner) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner> aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 463646775279.dkr.ecr.us-east-1.amazonaws.com
Login Succeeded
(vacation-planner) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner>
```

- Push the docker image

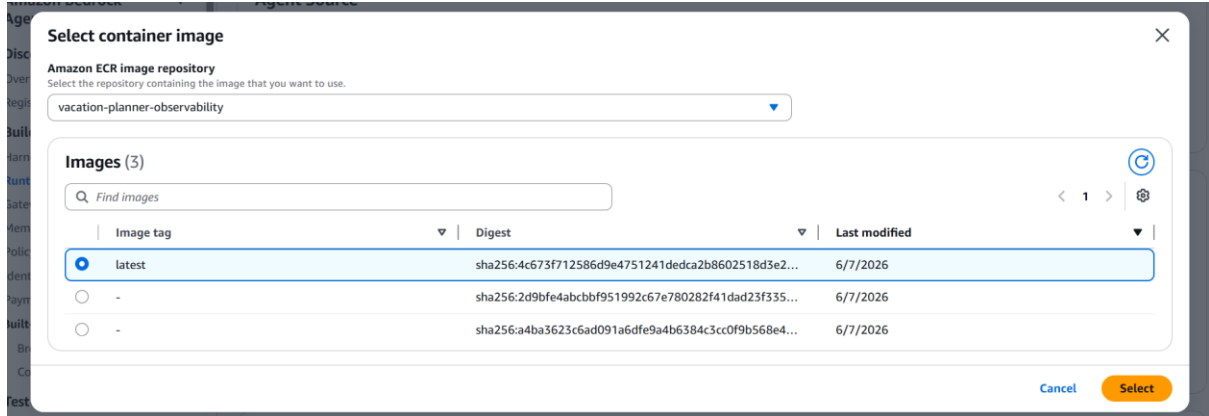
```
docker buildx build --platform linux/arm64 -t 463646775279.dkr.ecr.us-east-1.amazonaws.com/vacation-planner-observability:latest --push .
```

```
(vacation-planner) PS C:\Users\amanr\Downloads\Vacation Planner\vacation_planner> docker buildx build --platform linux/arm64 -t 463646775279.dkr.ecr.us-east-1.amazonaws.com/vacation-planner-observability:latest --push .
[+] Building 2.7s (0/1)
=> [internal] booting buildkit
=> => pulling image moby/buildkit:buildx-stable-1
=> => creating container buildx_buildkit_epic_goldstine0
docker-container:epic_goldstine
2.7s
2.6s
0.2s
```

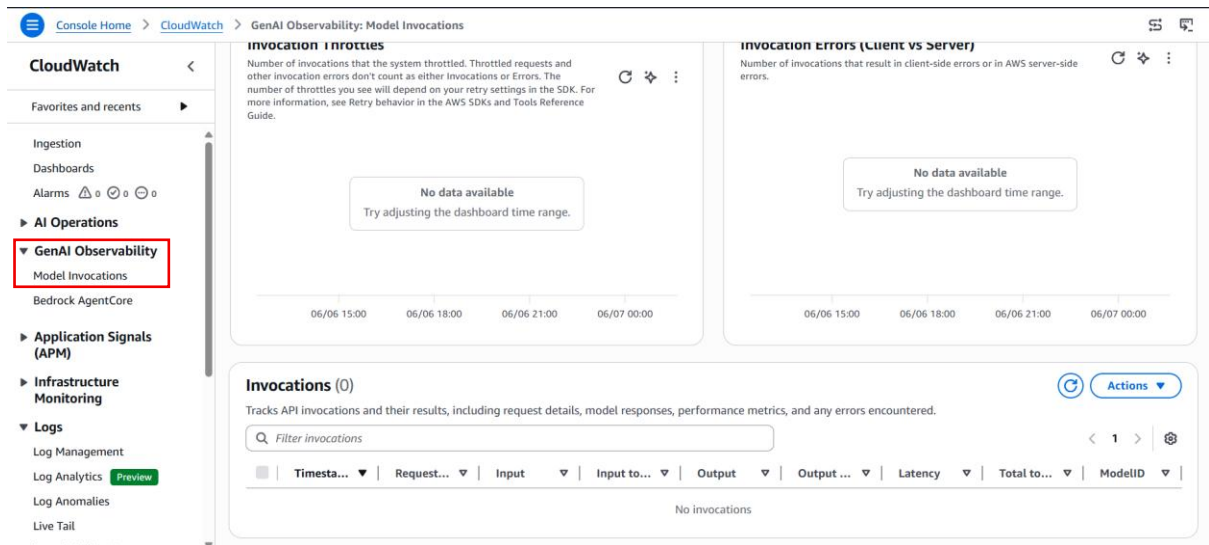
- After the image got deployed
- Go to the Agentcore to update the runtime
- Click Update hosting



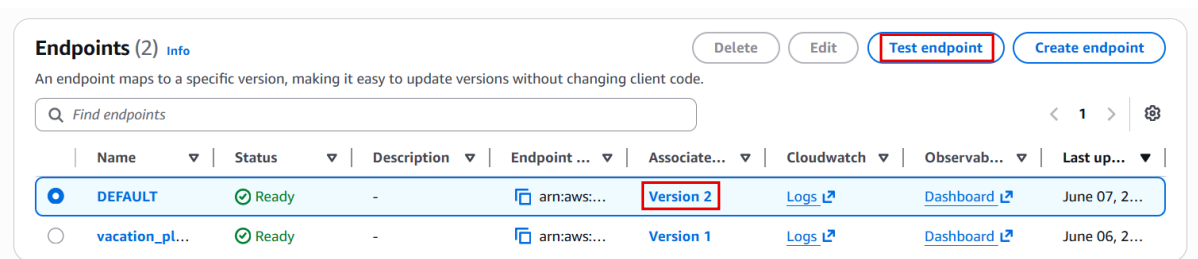
- Choose the image for the current docker image



- Click Host agent/tool
- Now go to CloudWatch > GenAI Observability > Model Invocation
- You will see no data.



- Now go back to Bedrock AgentCore
- Select the latest endpoint
- And test it





- Give the prompt

```
1 * {
2 "topic": "Plan a vacation to Bali, Indonesia"
3 }
```

- After running the prompt wait for few minutes and you will see the data
- Monitor key usage metrics, including invocation count, request volume, latency, token consumption (input/output), daily usage trends, and model-wise request distribution.

